

Documentação da Rota de Perfil do Usuário

Visão geral

A rota de perfil permite que o usuário autenticado:

- consulte seus dados completos;
- atualize informações cadastrais;
- altere o e-mail de login;
- envie ou troque a foto de perfil;
- altere a própria senha, quando for administrador.

Nesta documentação, a rota será considerada como:

`/api/perfil`

Caso o arquivo esteja em outro caminho, basta substituir o endpoint no documento.

A implementação usa autenticação via Supabase, valida o usuário na tabela `usuarios` e utiliza um cliente administrativo com `SUPABASE_SERVICE_ROLE_KEY` para operações sensíveis, como alteração de senha, atualização de e-mail no Auth e manipulação do avatar no Storage.

Endpoints disponíveis

Método	Endpoint	Finalidade
GET	<code>/api/perfil</code>	Retorna o perfil completo do usuário logado

`PUT` `/api/profile` Atualiza dados do perfil, senha e avatar

Variáveis de ambiente utilizadas

A rota depende das seguintes variáveis:

`NEXT_PUBLIC_SUPABASE_URL=`

`SUPABASE_SERVICE_ROLE_KEY=`

Finalidade

`NEXT_PUBLIC_SUPABASE_URL`

URL do projeto Supabase.

`SUPABASE_SERVICE_ROLE_KEY`

Chave administrativa usada para:

- atualizar usuários no Supabase Auth;
- alterar e-mail de login;
- alterar senha;
- enviar e excluir arquivos no bucket de avatares.

Essa chave deve permanecer **exclusivamente no backend**. Nunca deve ser exposta no front-end.

Autenticação e validação do perfil

Antes de executar qualquer operação, a rota chama:

`getAuthenticatedProfile()`

Essa função realiza três verificações principais.

1. Usuário autenticado

A sessão é validada com:

```
supabase.auth.getUser()
```

Se não houver sessão válida:

```
{  
  "error": "Usuário não autenticado."  
}
```

Status:

401 Unauthorized

2. Perfil existente

A rota consulta a tabela:

```
usuarios
```

buscando:

```
.eq("id", user.id)
```

Se o perfil não existir:

```
{  
  "error": "Perfil de usuário não encontrado."  
}
```

Status:

404 Not Found

3. Usuário ativo

A rota exige:

```
profile.status === "ativo"
```

Caso contrário:

```
{  
  "error": "Seu acesso está inativo ou bloqueado."  
}
```

Status:

403 Forbidden

Cliente administrativo

A rota possui um helper:

```
createAdminClient()
```

Ele cria um cliente Supabase com a chave:

```
SUPABASE_SERVICE_ROLE_KEY
```

Configuração utilizada:

```
auth: {  
  autoRefreshToken: false,  
  persistSession: false,  
}
```

Isso é adequado para uso backend, porque esse cliente não precisa manter sessão persistida nem renovar token automaticamente.

GET /api/perfil

Objetivo

Retornar o perfil completo do usuário autenticado.

A rota:

1. valida a sessão;
2. carrega o perfil autenticado;

3. consulta novamente o perfil usando o cliente administrativo;
4. garante que o perfil pertence à mesma `conta_id`;
5. retorna os dados seguros para o front-end.

Consulta executada

```
const { data: perfil, error: perfilError } = await admin
  .from("usuarios")
  .select("*")
  .eq("id", profile.id)
  .eq("conta_id", profile.conta_id)
  .single();
```

A dupla validação por:

`id`

`conta_id`

protege contra leitura indevida entre contas.

Resposta de sucesso

Status:

200 OK

Body:

```
{
  "success": true,
  "perfil": {
    "...": "dados completos do usuário",
    "pode_alterar_senha": true
  }
}
```

Campo adicional: `pode_alterar_senha`

Antes de enviar o perfil ao front-end, a rota aplica:

```
buildSafePerfil(perfil)
```

Essa função adiciona:

```
pode_alterar_senha: perfil.role === "admin"
```

Ou seja:

- administradores recebem `true`;
- demais perfis recebem `false`.

Isso facilita controlar a exibição do bloco de alteração de senha no front-end.

Erros possíveis no GET

Usuário não autenticado

```
{  
  "error": "Usuário não autenticado."  
}
```

Status:

```
401
```

Perfil inexistente

```
{  
  "error": "Perfil de usuário não encontrado."  
}
```

Status:

```
404
```

Usuário inativo

```
{  
  "error": "Seu acesso está inativo ou bloqueado."  
}
```

Status:

403

Falha ao carregar dados do perfil

```
{  
  "error": "Não foi possível carregar o perfil do usuário."  
}
```

Status:

500

Erro inesperado

```
{  
  "error": "Erro interno ao carregar o perfil."  
}
```

Status:

500

PUT /api/perfil

Objetivo

Atualizar os dados do perfil autenticado.

A rota aceita:

- JSON normal;
 - `multipart/form-data`, usado quando há envio de avatar.
-

Formatos aceitos no PUT

1. JSON

Exemplo:

```
{  
  "nome_completo": "Maria Eduarda de Paula",  
  "email": "duda@email.com",  
  "telefone": "(32) 99999-9999"  
}
```

2. FormData

Quando existir upload de foto, o body deve ser enviado como:

perfil: `JSON.stringify(perfilCompleto)`

avatar: arquivo opcional

Exemplo conceitual:

```
const formData = new FormData();  
  
formData.append(  
  "perfil",  
  JSON.stringify({  
    nome_completo: "Maria Eduarda de Paula",  
    email: "duda@email.com",  
    telefone: "(32) 99999-9999",
```



```
})
```

```
);
```

```
formData.append("avatar", file);
```

Leitura do body

A rota usa:

```
readRequestBody(request)
```

Essa função identifica automaticamente:

- `multipart/form-data`;
- JSON comum.

No caso de FormData, ela lê:

```
perfil
```

```
avatar
```

Se o JSON contido em `perfil` for inválido:

```
{  
  
  "error": "Payload do perfil inválido."  
  
}
```

Status:

```
400 Bad Request
```

Limpeza e normalização do payload

Antes de salvar, o conteúdo recebido passa por:

```
normalizePerfilPayload(body)
```

Dados removidos do body

A função impede atualização direta de campos sensíveis ou controlados pelo sistema:

id

conta_id

role

cargo

status

ultimo_acesso_em

created_at

updated_at

password

confirmPassword

pode_alterar_senha

image_url

Isso evita que o front envie acidentalmente, ou tente alterar, informações que não devem ser editadas nessa tela.

Normalizações aplicadas

Nome completo

nome_completo: normalizeText(...)

Remove espaços em excesso e converte texto vazio em `null`.

E-mail

email: normalizeEmail(...)

- remove espaços;
- converte para minúsculas;
- retorna `null` se estiver vazio.

Telefone

telefone: normalizeText(...)

Validações obrigatórias no PUT

Nome completo

Se não vier preenchido:

```
{  
  "error": "Informe o nome completo."  
}
```

Status:

400 Bad Request

E-mail

Se não vier preenchido:

```
{  
  "error": "Informe o e-mail."  
}
```

Status:

400 Bad Request

Atualização de senha

Regra de negócio

A troca de senha nessa rota é permitida apenas quando:

```
perfilAtual.role === "admin"
```

Se um usuário não-admin tentar alterar senha por essa tela:

```
{  
  "error": "Apenas administradores podem alterar a senha nesta tela."  
}
```

Status:

403 Forbidden

Condições para disparar troca de senha

A lógica entra nesse bloco quando qualquer um destes campos existir:

password

confirmPassword

Regras de senha

Mínimo de 8 caracteres

```
{  
  "error": "A nova senha precisa ter pelo menos 8 caracteres."  
}
```

Status:

400 Bad Request

Confirmação deve coincidir

```
{  
  "error": "As senhas não coincidem."  
}
```

Status:

400 Bad Request

Atualização no Auth

Quando válida, a senha é atualizada com:

```
admin.auth.admin.updateUserById(profile.id, {  
  password,  
});
```

Se o Supabase falhar:

```
{  
  "error": "Não foi possível atualizar a senha."  
}
```

Status:

500 Internal Server Error

Atualização de e-mail

Objetivo

Manter sincronizados:

- e-mail da tabela `usuarios`;
 - e-mail usado no login do Supabase Auth.
-

Quando ocorre

A atualização é executada apenas se:

```
payload.email !== perfilAtual.email
```

Atualização no Auth

```
admin.auth.admin.updateUserById(profile.id, {  
  email: payload.email,  
  email_confirm: true,  
});
```

O uso de:

```
email_confirm: true
```

faz com que o e-mail atualizado já fique confirmado no Auth, sem exigir novo fluxo de confirmação.

Erro possível

```
{  
  "error": "Não foi possível atualizar o e-mail de login."  
}
```

Status:

```
500 Internal Server Error
```

Upload de avatar

Bucket utilizado

```
avatars
```

Tipos permitidos

A rota aceita apenas:

```
image/jpeg
```

```
image/png
```

```
image/webp
```

Tamanho máximo

5 MB

Erros possíveis

Arquivo inválido

```
{  
  "error": "Arquivo de avatar inválido."  
}
```

Tipo não permitido

```
{  
  "error": "Envie uma imagem JPG, PNG ou WEBP."  
}
```

Arquivo acima do limite

```
{  
  "error": "A imagem deve ter no máximo 5 MB."  
}
```

Status desses casos:

400 Bad Request

Caminho do avatar no Storage

O arquivo é salvo com padrão:

```
conta_id/usuario_id/uuid.extensao
```

Exemplo:

```
uuid-conta/uuid-usuario/7f2105d9-6b9a-4bd8-90f1-29d8ce.jpg
```

A rota monta esse caminho automaticamente por meio de:

```
buildAvatarStoragePath(profile, avatar)
```

Upload e URL pública

Após enviar o arquivo, a rota gera a URL pública com:

```
admin.storage
```

```
.from(AVATAR_BUCKET)
```

```
.getPublicUrl(avatarPath);
```

A URL resultante é salva em:

```
usuarios.image_url
```

Substituição de avatar antigo

Quando um novo avatar é enviado:

1. a rota sobe o novo arquivo;
2. atualiza a tabela `usuarios` com a nova URL;
3. somente depois remove o avatar anterior.

Essa ordem é excelente, porque evita apagar a foto antiga antes de garantir que a nova foi salva com sucesso.

Atualização final do perfil

A atualização usa:

```
.update({
```



```
...payload,  
  
image_url: nextImageUrl,  
  
updated_at: new Date().toISOString(),  
  
})  
  
.eq("id", profile.id)  
  
.eq("conta_id", profile.conta_id)  
  
.select("*")  
  
.single();
```

Assim, a rota:

- atualiza apenas o usuário logado;
 - respeita a conta vinculada;
 - devolve o perfil já atualizado.
-

Resposta de sucesso no PUT

Sem troca de avatar

```
{  
  
  "success": true,  
  
  "message": "Perfil atualizado com sucesso.",  
  
  "perfil": {  
  
    "...": "dados atualizados"  
  
  }  
  
}
```

Com troca de avatar

```
{  
  
  "success": true,
```

```
"message": "Perfil e foto atualizados com sucesso.",
"perfil": {
  "...": "dados atualizados"
}
}
```

Erros possíveis no PUT

Situação	Status	Resposta
	400	
Sem autenticação	401	Usuário não autenticado.
Perfil inexistente	404	Perfil de usuário não encontrado.
Usuário inativo	403	Seu acesso está inativo ou bloqueado.
Nome ausente	400	Informe o nome completo.
E-mail ausente	400	Informe o e-mail.

Payload inválido	40	Payload do perfil inválido.
Avatar inválido	40	Mensagem específica
Tentativa indevida de trocar senha	40	Apenas admins podem alterar a senha
Senha curta	40	Mínimo de 8 caracteres
Senhas diferentes	40	As senhas não coincidem
Erro ao trocar senha	50	Não foi possível atualizar a senha
Erro ao alterar e-mail	50	Não foi possível atualizar o e-mail
Erro ao enviar avatar	50	Não foi possível enviar a nova foto
Erro ao salvar perfil	50	Não foi possível atualizar o perfil
Erro inesperado	50	Erro interno ao atualizar o perfil

Fluxo completo do GET

1. Recebe GET /api/perfil
 2. Valida sessão do usuário
 3. Busca perfil em usuarios
 4. Confirma status ativo
 5. Cria cliente administrativo
 6. Busca perfil completo por id + conta_id
 7. Adiciona pode_alterar_senha
 8. Retorna perfil ao front-end
-

Fluxo completo do PUT

1. Recebe PUT /api/perfil
2. Valida sessão do usuário
3. Busca perfil em usuarios
4. Confirma status ativo
5. Lê body JSON ou FormData
6. Remove campos proibidos
7. Normaliza nome, e-mail e telefone
8. Valida nome e e-mail
9. Valida avatar, se existir
10. Busca perfil atual via admin
11. Atualiza senha, se solicitada e permitida
12. Atualiza e-mail no Supabase Auth, se alterado
13. Faz upload do novo avatar, se enviado
14. Atualiza tabela usuarios

15. Remove avatar antigo, se necessário

16. Retorna perfil atualizado

Estrutura resumida da rota

Item	Definição
Endpoint	<code>/api/perfil</code>
Métodos	<code>GET, PUT</code>
Autenticação	Obrigatória
Consulta de perfil	Tabela <code>usuarios</code>
Atualização de perfil	Tabela <code>usuarios</code>
Atualização de senha	Supabase Auth Admin
Atualização de e-mail	Supabase Auth Admin
Upload de avatar	Supabase Storage
Bucket	<code>avatars</code>

Tipos aceitos	JPG, PNG, WEBP
---------------	----------------

Tamanho máximo	5 MB
----------------	------

JSON aceito	Sim
-------------	-----

FormData aceito	Sim
-----------------	-----

Proteção multi-conta	<code>id + conta_id</code>
-------------------------	----------------------------

